

Python Quick Reference Guide

Basic Syntax

```
# Comments
x = 5; name = "Alice"; is_valid = True
a, b, c = 1, 2, 3 # Multiple assignment
x = y = z = 0
```

Data Types

```
int_num = 42; float_num = 3.14
text = "Hello"; f_string = f"Hi {name}"
is_true = True; empty = None
```

String Methods

```
s = "Hello World"
s.upper(), s.lower(), s.strip()
s.replace("o", "0"), s.split()
s.startswith("He"), s.endswith("ld")
len(s), s[0], s[-1], s[1:4]
```

Lists

```
lst = [1, 2, 3]; mixed = [1, "hi", 3.14]
lst.append(4), lst.insert(0, 0)
lst.remove(2), lst.pop(), lst.extend([5,6])
lst.sort(), lst.reverse()
first = lst[0], last = lst[-1]
slice = lst[1:3], length = len(lst)
```

List Comprehensions

```
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x%2==0]
matrix = [[i*j for j in range(3)] for i in range(3)]
```

Dictionaries

```
person = {"name": "Alice", "age": 25}
person["email"] = "alice@email.com"
age = person["age"]
age = person.get("age", 0) # Safe access
del person["age"]
keys = person.keys(), values = person.values()
items = person.items()
```

Tuples & Sets

```
coords = (10, 20); x, y = coords
unique = {1, 2, 3, 3} # {1, 2, 3}
unique.add(4), unique.remove(1)
```

Control Flow

```
if x > 0:
    print("positive")
elif x < 0:
    print("negative")
else:
    print("zero")

result = "pos" if x > 0 else "not pos"
```

Loops

```
for i in range(5): # 0 to 4
    print(i)

for i in range(2, 10, 2): # 2,4,6,8
    print(i)

for item in [1, 2, 3]:
    print(item)

for k, v in person.items():
    print(f"{k}: {v}")

count = 0
while count < 5:
    count += 1

for i in range(10):
    if i == 3: continue
    if i == 7: break
```

Functions

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

def add(a, b):
    return a + b

def sum_all(*args):
    return sum(args)

def create_person(**kwargs):
    return kwargs
```

```
square = lambda x: x**2
add = lambda x, y: x + y
```

File I/O

```
with open("file.txt", "r") as f:
    content = f.read()
    lines = f.readlines()

with open("file.txt", "w") as f:
    f.write("Hello World")

with open("file.txt", "a") as f:
    f.write("Appended")
```

Error Handling

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:
    print(f"Error: {e}")
else:
    print("No errors")
finally:
    print("Always runs")

raise ValueError("Custom error")
```

Classes

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hi, I'm {self.name}"

person = Person("Alice", 25)

class Student(Person):
    def __init__(self, name, age, id):
        super().__init__(name, age)
        self.student_id = id
```

Built-in Functions

```
abs(-5), min(1,2,3), max(1,2,3)
round(3.7), pow(2,3), len([1,2,3])
sum([1,2,3]), sorted([3,1,2])
int("123"), float("3.14"), str(123)
list("abc"), tuple([1,2,3])
enumerate(['a','b']), zip([1,2],[ 'a','b'])
all([True,False]), any([True,False])
```

Imports

```
import math
from math import sqrt, pi
import numpy as np
from datetime import datetime

if __name__ == "__main__":
    main()
```

String Formatting

```
name, age = "Alice", 25
f"I'm {name}, age {age}"
"I'm {}, age {}".format(name, age)
"I'm {name}, age {age}".format(**locals())
"{:.2f}".format(3.14159) # 3.14
```

Useful Patterns

```
a, b = b, a # Swap
if x in [1, 2, 3]: pass
if 0 <= x <= 100: pass
name = user_input or "Anonymous"
" ".join(["Hello", "World"])
count = counts.get(key, 0) + 1
```

Comprehensions

```
# Dict comprehension
{k:v for k,v in items if condition}
# Set comprehension
{x**2 for x in range(10)}
# Generator
(x**2 for x in range(10))
```

Advanced List Operations

```
# Slicing
lst[start:end:step]
lst[::-1] # Reverse
lst[::2] # Every 2nd

# List methods
lst.index(value), lst.count(value)
lst.clear(), lst.copy()

# Nested list flattening
flat = [item for sub in nested for item in sub]
```

Dictionary Operations

```
# Dictionary methods
d.update(other_dict)
d.pop(key, default)
d.popitem() # Remove last item
d.setdefault(key, default)

# Merge dictionaries (Python 3.9+)
merged = dict1 | dict2

# Dictionary from keys
dict.fromkeys(['a', 'b', 'c'], 0)
```

Set Operations

```
s1, s2 = {1,2,3}, {3,4,5}
s1.union(s2), s1 | s2
s1.intersection(s2), s1 & s2
s1.difference(s2), s1 - s2
s1.symmetric_difference(s2), s1 ^ s2
s1.issubset(s2), s1.issuperset(s2)
s1.isdisjoint(s2)
```

Regular Expressions

```
import re
re.search(r'pattern', text)
re.match(r'pattern', text)
re.findall(r'pattern', text)
re.sub(r'pattern', 'replacement', text)
re.split(r'pattern', text)

# Common patterns
r'\d+' # Digits
r'\w+' # Word chars
r'\s+' # Whitespace
r'[a-zA-Z]+' # Letters
```

Datetime

```
from datetime import datetime, date, timedelta
now = datetime.now()
today = date.today()
tomorrow = today + timedelta(days=1)
now.strftime("%Y-%m-%d %H:%M:%S")
datetime.strptime("2023-01-01", "%Y-%m-%d")

# Common format codes
%Y year, %m month, %d day
%H hour, %M minute, %S second
```

Math Module

```
import math
math.sqrt(16), math.pow(2, 3)
math.ceil(3.1), math.floor(3.9)
math.sin(math.pi), math.cos(0)
math.log(10), math.log10(100)
math.factorial(5)
math.gcd(12, 18)
math.pi, math.e
```

Random Module

```
import random
random.random() # 0.0 to 1.0
random.randint(1, 10)
random.choice(['a', 'b', 'c'])
random.sample([1,2,3,4], 2)
random.shuffle(my_list)
random.seed(42) # Reproducible
```

OS Module

```
import os
os.getcwd(), os.chdir('/path')
os.listdir('.'), os.mkdir('folder')
os.path.exists('file'), os.path.isfile('f')
os.path.isdir('d'), os.path.join('a', 'b')
os.path.basename('/a/b/c.txt') # c.txt
os.path.dirname('/a/b/c.txt') # /a/b
os.environ['HOME']
```

JSON Module

```
import json
data = {"name": "Alice", "age": 25}
json_str = json.dumps(data)
parsed = json.loads(json_str)

# File operations
with open('data.json', 'w') as f:
    json.dump(data, f)
with open('data.json', 'r') as f:
    data = json.load(f)
```

Itertools

```
from itertools import *
count(10, 2) # 10, 12, 14...
cycle(['A', 'B']) # A, B, A, B...
repeat('hi', 3) # hi, hi, hi
combinations([1,2,3], 2)
permutations([1,2,3], 2)
product([1,2], ['A', 'B'])
chain([1,2], [3,4]) # 1, 2, 3, 4
```

Collections

```
from collections import *
Counter(['a', 'b', 'a', 'c']) # {'a':2, 'b':1, 'c':1}
defaultdict(int) # Default value 0
deque([1,2,3]) # Double-ended queue
OrderedDict() # Ordered dictionary
namedtuple('Point', ['x', 'y'])
```

Functional Programming

```
# Map, filter, reduce
list(map(lambda x: x**2, [1,2,3]))
list(filter(lambda x: x%2==0, [1,2,3,4]))
from functools import reduce
reduce(lambda x,y: x+y, [1,2,3,4])

# Decorators
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before")
        result = func(*args, **kwargs)
        print("After")
        return result
    return wrapper

@my_decorator
def greet(): print("Hello")
```

Context Managers

```
# Custom context manager
from contextlib import contextmanager
@contextmanager
def my_context():
    print("Enter")
    try:
        yield "resource"
    finally:
        print("Exit")

with my_context() as resource:
    print(f"Using {resource}")
```

Generators

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
next(fib), next(fib), next(fib)

# Generator expression
squares = (x**2 for x in range(10))
```

Property Decorators

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("Negative radius")
        self._radius = value
```

Magic Methods

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self): return f"({self.x}, {self.y})"
    def __repr__(self): return f"Point({self.x}, {self.y})"
    def __add__(self, other):
        return Point(self.x+other.x, self.y+other.y)
    def __eq__(self, other):
        return self.x==other.x and self.y==other.y
```

Type Hints (Python 3.5+)

```
from typing import List, Dict, Optional, Union

def greet(name: str) -> str:
    return f"Hello, {name}"

def process(items: List[int]) -> Dict[str, int]:
    return {"count": len(items)}

def find(lst: List[str], item: str) -> Optional[int]:
    try:
        return lst.index(item)
    except ValueError:
        return None

age: int = 25
name: Optional[str] = None
data: Union[str, int] = "hello"
```

Pathlib (Python 3.4+)

```
from pathlib import Path

p = Path("folder/file.txt")
p.exists(), p.is_file(), p.is_dir()
p.name, p.suffix, p.stem
p.parent, p.parents
p.absolute(), p.resolve()

# Create directories
Path("new/dir").mkdir(parents=True, exist_ok=True)

# Read/write files
p.read_text(), p.write_text("content")
p.read_bytes(), p.write_bytes(b"data")
```

F-strings Advanced

```
name, score = "Alice", 95.567

f"{name:>10}" # Right align
f"{name:<10}" # Left align
f"{name:^10}" # Center
f"{score:.2f}" # 2 decimal places
f"{score:08.2f}" # Zero padding
f"{1000:}," # Comma separator
f"{0.15:.1%}" # Percentage

# Debug (Python 3.8+)
f"{name=}" # name='Alice'
```

Walrus Operator (Python 3.8+)

```
# Assignment in expressions
if (n := len(items)) > 10:
    print(f"Too many: {n}")

while (line := file.readline()):
    process(line)

# In comprehensions
[y for x in data if (y := func(x)) > 0]
```

Data Classes (Python 3.7+)

```
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int
    email: str = ""

person = Person("Alice", 25)
print(person) # Auto __str__
```

Enum (Python 3.4+)

```
from enum import Enum, auto

class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3

class Status(Enum):
    PENDING = auto()
    PROCESSING = auto()
    COMPLETE = auto()

color = Color.RED
print(color.name, color.value)
```

Match Statement (Python 3.10+)

```
def handle_data(data):
    match data:
        case int() if data > 0:
            return "positive integer"
        case str() if len(data) > 10:
            return "long string"
        case [first, *rest]:
            return f"list: {first}, {rest}"
        case {"name": str(name), "age": int(age)}:
            return f"{name} is {age}"
        case _:
            return "unknown"
```

Async/Await (Python 3.7+)

```
import asyncio

async def fetch_data(url):
    await asyncio.sleep(1) # Simulate I/O
    return f"Data from {url}"

async def main():
    tasks = [fetch_data(f"url{i}") for i in range(3)]
    results = await asyncio.gather(*tasks)
    return results

# Run async function
asyncio.run(main())
```

Testing with unittest

```
import unittest

class TestMath(unittest.TestCase):
    def test_addition(self):
        self.assertEqual(2 + 2, 4)
        self.assertNotEqual(2 + 2, 5)

    def test_division(self):
        with self.assertRaises(ZeroDivisionError):
            10 / 0

if __name__ == "__main__":
    unittest.main()
```

Logging

```
import logging

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')

logger = logging.getLogger(__name__)
logger.debug("Debug message")
logger.info("Info message")
logger.warning("Warning message")
logger.error("Error message")
logger.critical("Critical message")
```

Performance & Profiling

```
import time
import cProfile
from timeit import timeit

# Timing code
start = time.time()
# ... code ...
elapsed = time.time() - start

# Timeit for small snippets
timeit('sum([1,2,3,4,5])', number=100000)

# Profiling
cProfile.run('expensive_function()')
```

Memory Management

```
import gc
import sys

# Garbage collection
gc.collect() # Force collection
gc.disable(), gc.enable()

# Memory usage
sys.getsizeof(object)
sys.getrefcount(object)

# Weak references
import weakref
weak_ref = weakref.ref(object)
```

Virtual Environments

```
# Command line
python -m venv myenv
source myenv/bin/activate # Linux/Mac
myenv\Scripts\activate # Windows
deactivate

pip install package
pip install -r requirements.txt
pip freeze > requirements.txt
pip list, pip show package
```

Debugging

```
import pdb

# Set breakpoint
pdb.set_trace()
# Python 3.7+
breakpoint()

# PDB commands
# n - next line
# s - step into
# c - continue
# l - list source
# p var - print variable
# q - quit
```

Common Gotchas

```
# Mutable default arguments
def func(lst=[]): # DON'T
    lst.append(1)

def func(lst=None): # DO
    if lst is None:
        lst = []

# Late binding closures
funcs = [lambda i for i in range(3)] # All return 2
funcs = [lambda x=i: x for i in range(3)] # Fix

# Chained comparisons
False == False in [False] # True (not False)
```

Best Practices

```
# PEP 8 style guide
# - 4 spaces for indentation
# - snake_case for functions/variables
# - PascalCase for classes
# - UPPER_CASE for constants
# - Max line length: 79 characters

# Use is/is not for None
if value is None: pass

# Use truthiness
if items: pass # instead of len(items) > 0
```

Common Algorithms

```
# Binary search
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Quick sort
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)
```

Data Structures

```
# Stack (LIFO)
stack = []
stack.append(item) # push
item = stack.pop() # pop

# Queue (FIFO)
from collections import deque
queue = deque()
queue.append(item) # enqueue
item = queue.popleft() # dequeue

# Priority Queue
import heapq
heap = []
heapq.heappush(heap, (priority, item))
priority, item = heapq.heappop(heap)
```

File Formats

```
# CSV
import csv
with open('data.csv', 'r') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['column'])

with open('out.csv', 'w') as f:
    writer = csv.writer(f)
    writer.writerow(['a', 'b', 'c'])

# XML
import xml.etree.ElementTree as ET
tree = ET.parse('data.xml')
root = tree.getroot()
for child in root:
    print(child.tag, child.text)
```

Web Requests

```
# HTTP requests (external library)
import requests

response = requests.get('https://api.github.com')
data = response.json()
status = response.status_code

response = requests.post('https://api.com',
    json={'key': 'value'},
    headers={'Authorization': 'Bearer token'})

# Built-in urllib
from urllib.request import urlopen
with urlopen('https://example.com') as response:
    data = response.read()
```

Database (SQLite)

```
import sqlite3

# Connect and create table
conn = sqlite3.connect('example.db')
cursor = conn.cursor()

cursor.execute('''CREATE TABLE users
                  (id INTEGER PRIMARY KEY,
                   name TEXT NOT NULL)''')

# Insert data
cursor.execute("INSERT INTO users (name)
VALUES (?)", ('Alice',))
conn.commit()

# Query data
cursor.execute("SELECT * FROM users")
rows = cursor.fetchall()

conn.close()
```

Command Line Arguments

```
import sys
import argparse

# Simple sys.argv
script_name = sys.argv[0]
args = sys.argv[1:]

# Argparse
parser = argparse.ArgumentParser()
parser.add_argument('filename')
parser.add_argument('--verbose', '-v',
                    action='store_true')
parser.add_argument('--count', '-c',
                    type=int, default=1)
args = parser.parse_args()
```

Environment Variables

```
import os

# Read environment variables
home = os.environ['HOME']
path = os.environ.get('PATH', '')
debug = os.environ.get('DEBUG', 'False').lower() == 'true'

# Set environment variables
os.environ['MY_VAR'] = 'value'

# Load from .env file (external library)
from dotenv import load_dotenv
load_dotenv()
```

Multiprocessing

```
from multiprocessing import Pool, Process

def worker(x):
    return x * x

# Process pool
with Pool(4) as pool:
    results = pool.map(worker, [1,2,3,4,5])

# Individual processes
p = Process(target=worker, args=(5,))
p.start()
p.join()
```

Threading

```
import threading
import time

def worker(name):
    print(f"Starting {name}")
    time.sleep(2)
    print(f"Finished {name}")

# Create and start threads
threads = []
for i in range(3):
    t = threading.Thread(target=worker, args=(f"Thread-{i}",))
    t.start()
    threads.append(t)

# Wait for completion
for t in threads:
    t.join()
```

Package Management

```
# pip commands
pip install package
pip install package==1.2.3
pip install -r requirements.txt
pip install -e . # Editable install
pip uninstall package
pip list
pip show package
pip freeze > requirements.txt
pip search package

# setup.py
from setuptools import setup
setup(
    name='mypackage',
    version='0.1.0',
    packages=['mypackage'],
    install_requires=['requests']
)
```

Creating Modules

```
# mymodule.py
def my_function():
    return "Hello from module"

MY_CONSTANT = 42

if __name__ == "__main__":
    print("Module run directly")

# __init__.py (package)
from .module1 import function1
from .module2 import function2

__all__ = ['function1', 'function2']
__version__ = '1.0.0'
```

Security

```
import hashlib
import secrets

# Hashing
hash_object = hashlib.sha256(b'hello world')
hex_dig = hash_object.hexdigest()

# Secure random
token = secrets.token_urlsafe(32)
password = secrets.token_hex(16)

# Compare strings securely
secrets.compare_digest(a, b)
```

Common One-liners

```
# Flatten list
[item for sublist in nested for item in sublist]

# Remove duplicates preserving order
list(dict.fromkeys(items))

# Most common element
max(set(items), key=items.count)

# Transpose matrix
list(zip(*matrix))

# Check if all elements are unique
len(items) == len(set(items))

# Find indices of element
[i for i, x in enumerate(items) if x == target]
```

Performance Tips

```
# Use list comprehensions
[x*2 for x in range(1000)] # Fast
# vs for loop append # Slower

# Use sets for membership testing
if item in my_set: # O(1)
# vs if item in my_list # O(n)

# Use enumerate instead of range(len())
for i, item in enumerate(items): # Good
# vs for i in range(len(items)) # Bad

# Use collections.Counter
from collections import Counter
counts = Counter(items) # Fast counting
```